Shade System

Students: Amit Simona, Michael Yehoshua, Ofir Duek, Guy Cohen

Moderator: Eliel Yeshayahu

Table of contents

3 The problem

4 The Current (Inefficient) Solution

5 Our Solution

6 Main Processes

7 Main modules

14 Requirements from the specification document

15 Main products - Screenshots

24 Unexpected problems

25 Copin and Solving

27 Tecnologies used

28 Conclusions for further developments

The problem

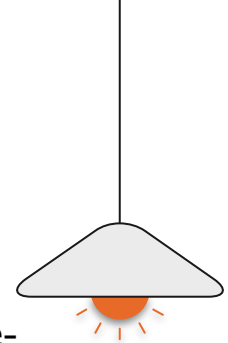
Managing shaded areas and outdoor spaces across campuses is currently inefficient and unorganized. Information is scattered between Excel files, photos, WhatsApp messages, and emails, with no single source of truth. There is no standardized way to mark shade locations, track maintenance dates, or record coverage levels.

→ This leads to inconsistent data, delays in field operations, and unnecessary time and cost wasted on repetitive tasks

The Current (Inefficient) Solution

Today, most campuses rely on manual coordination between departments. Updates are shared through spreadsheets, images, and informal communication channels. There is no unified platform to view the full picture or track changes over time. As a result, planning, maintenance, and decision-making are based on outdated or incomplete information.

Our Solution



We are developing a **web-based management platform** that centralizes all shade-related data in one place.
The system provides:

- Adding a new area
- An interactive map for visualizing and locating shade structures
- Digital records of dimensions, coverage
- User roles and permissions for secure collaboration

Our platform transforms shade management from a manual and fragmented process into a **smart, data-driven, and transparent system** that saves time, improves coordination, and enhances the outdoor experience across campus.

Main Processes

- **New / Existing Areas Support:**
  Users can upload a new area map or use existing ones to manage the shades placement in it.
- **Shade Uploading:**
  Users are able to upload new shades and place them at their correct location on the map.
- **Shade Editing and Removal:**
  Users have the option to edit existing shades properties or removing them whenever and however they need.
- **Administrator Functions:**
  Administrators have permissions to add new users with specific roles (administrator,viewer). They can delete users from the system and update user permissions as needed.

Main modules - Starting the server

- **Load Express and Middleware**
  The server loads Express and middleware to handle HTTP requests and responses.
- **Connect to the Database**
  The server connects to the MySQL database to perform CRUD operations.
- **Start the Server**
  The server starts and listens for incoming HTTP requests.

```
const express = require("express");
app.use(cors());
app.use(express.json());

// ================= connect  DB =================
const db = require("./db"); // Pool של MySQL2
// ================= start server =================
app.listen(PORT, () => {
  console.log(` השרת רץ על http://localhost:${PORT}`);
});
```

Main modules - Controllers

- The route handler receives the HTTP request and extracts the relevant data from the request body or parameters.

- It interacts with the database to perform the required operation (such as login, registration, data retrieval, update, or deletion).

- The handler returns an appropriate HTTP response:
  - 200 if the operation was successful
  - 400 if there was a problem with the request
  - 401 if the user is not authorized
  - 500 if there was a server error

```
// ================ login/register ================
app.post("/api/login", async (req, res) => {
  const { username, password } = req.body;
  if (!username || !password)
    return res.status(400).json({ error: "Username and password required" });

  try {
    const [results] = await db.query(
      "SELECT * FROM users WHERE username = ? AND password = ?",
      [username, password]
    );
    if (results.length > 0) {
      res.json({ success: true });
    } else {
      res.status(401).json({ success: false, error: "Invalid credentials" });
    }
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

Main modules - Models

- The model interacts directly with the database to read, write, update, or delete data.
- It performs data validation before saving to the database.
- The model handles any extra logic needed for the request.

```
app.post("/api/register", async (req, res) => {
  let username = (req.body?.username ?? "").toString().trim();
  let password = (req.body?.password ?? "").toString().trim();
  let email = (req.body?.email ?? "").toString().trim().toLowerCase();
  let phone = (req.body?.phone ?? "").toString().trim();

  if (!username) return res.status(400).json({ error: "חובה למלא שם משתמש" });
  if (!password) return res.status(400).json({ error: "חובה למלא סיסמה" });
  if (!email) return res.status(400).json({ error: "חובה למלא כתובת מייל" });
  if (!phone) return res.status(400).json({ error: "חובה למלא מספר טלפון" });

  // Email validation: must contain @ and then a dot after it
  const emailRegex = /^[^@\s]+@[^@\s]+\.[^@\s]+$/;
  if (!emailRegex.test(email)) {
    return res.status(400).json({ error: "כתובת מייל אינה חוקית" });
  }

  // Phone validation: exactly 10 digits
  const phoneRegex = /^\d{10}$/;
  if (!phoneRegex.test(phone)) {
    return res.status(400).json({ error: "מספר הפלאפון אינו חוקי" });
  }
```

Main modules - DB

- The database module manages the connection to the database and provides functions for executing queries.
- It allows the server to store, retrieve, and update information efficiently.

```
app.get("/api/areas", async (req, res) => {
  try {
    const [results] = await db.query("SELECT * FROM areas");
    res.json(results);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.get("/api/areas/name/:name", async (req, res) => {
  try {
    const areaName = decodeURIComponent(req.params.name);
    const [results] = await db.query("SELECT * FROM areas WHERE name = ?", [
      areaName,
    ]);
    if (results.length === 0)
      return res.status(404).json({ error: "אזור לא נמצא" });
    res.json(results[0]);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

Main modules - Bucket

- To allow users to upload files (such as images), the server uses a dedicated upload folder and handles file storage using Multer.
- The upload logic saves files with unique names and makes them accessible via a static route.

```
const app = express();
const PORT = 5000;

const UPLOADS_DIR = path.join(__dirname, "uploads");
if (!fs.existsSync(UPLOADS_DIR)) {
  fs.mkdirSync(UPLOADS_DIR, { recursive: true });
}

app.use("/uploads", express.static(UPLOADS_DIR));

// ================= file storage =================
const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, UPLOADS_DIR),
  filename: (req, file, cb) => {
    const name = file.originalname.replace(/\s+/g, "_");
    const time = new Date().toISOString().replace(/[:.]/g, "-");
    cb(null, `${time}-${name}`);
  },
});
const upload = multer({ storage });
```

Main modules - Config

- There are separate configuration files for different parts of the server, such as the database settings and other system parameters.

```
const mysql = require("mysql2/promise");
const pool = mysql.createPool({
  host: "localhost",
  user: "root",
  password: "abcd1234",
  database: "shade_system_db",
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0,
});

module.exports = pool;
```

Main modules - User management

- User-related logic is organized into separate files based on user roles and management operations.
- The API allows administrators to add, delete, update, search, and get users in the system.

```
app.post("/api/users/add", async (req, res) => {
  try {
    await db.query(
      "INSERT INTO users (username, password, email, role) VALUES (?, ?, ?, ?)",
      [req.body.username, req.body.password, req.body.email, req.body.role]
    );
    res.json({ success: true });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.delete("/api/users/:id", async (req, res) => {
  try {
    await db.query("DELETE FROM users WHERE id = ?", [req.params.id]);
    res.json({ success: true });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

Requirements from the specification document

- **Accessible shading information across campus -** The system presents clear, current data on shading locations so students and staff can quickly understand on-site conditions and use it in day-to-day activities and events.
- **Streamlined workflows and inter-department coordination -** By centralizing information in one place, the platform reduces repeated site visits and improves coordination among planning, maintenance, and administration units - enabling faster response and better service for students and staff.
- **Improved user experience and safety -** The platform supports informed planning of shaded areas for staying, learning, and activities - enhancing comfort and reducing sun exposure for the benefit of students and staff.
- **Data-driven decision-making -** A consolidated visual status by Area helps identify shading gaps and prioritize upgrades and maintenance according to real needs in the field.
- **Transparency and quality control -** Role-based access separates viewing from editing, preserving data consistency and accuracy while enabling change tracking - supporting service quality and clear communications with students and staff.
- **Innovative and sustainable image -** Demonstrates the use of advanced technology for managing outdoor spaces, projecting professionalism and innovation that strengthens the organization's brand. Creating comfortable shaded zones also reduces reliance on air-conditioning and supports responsible environmental practices.

Main Products - Screenshots

- Login Page

- Sign Up

- Areas management page

- Area’s Shades Control

- Add shade page

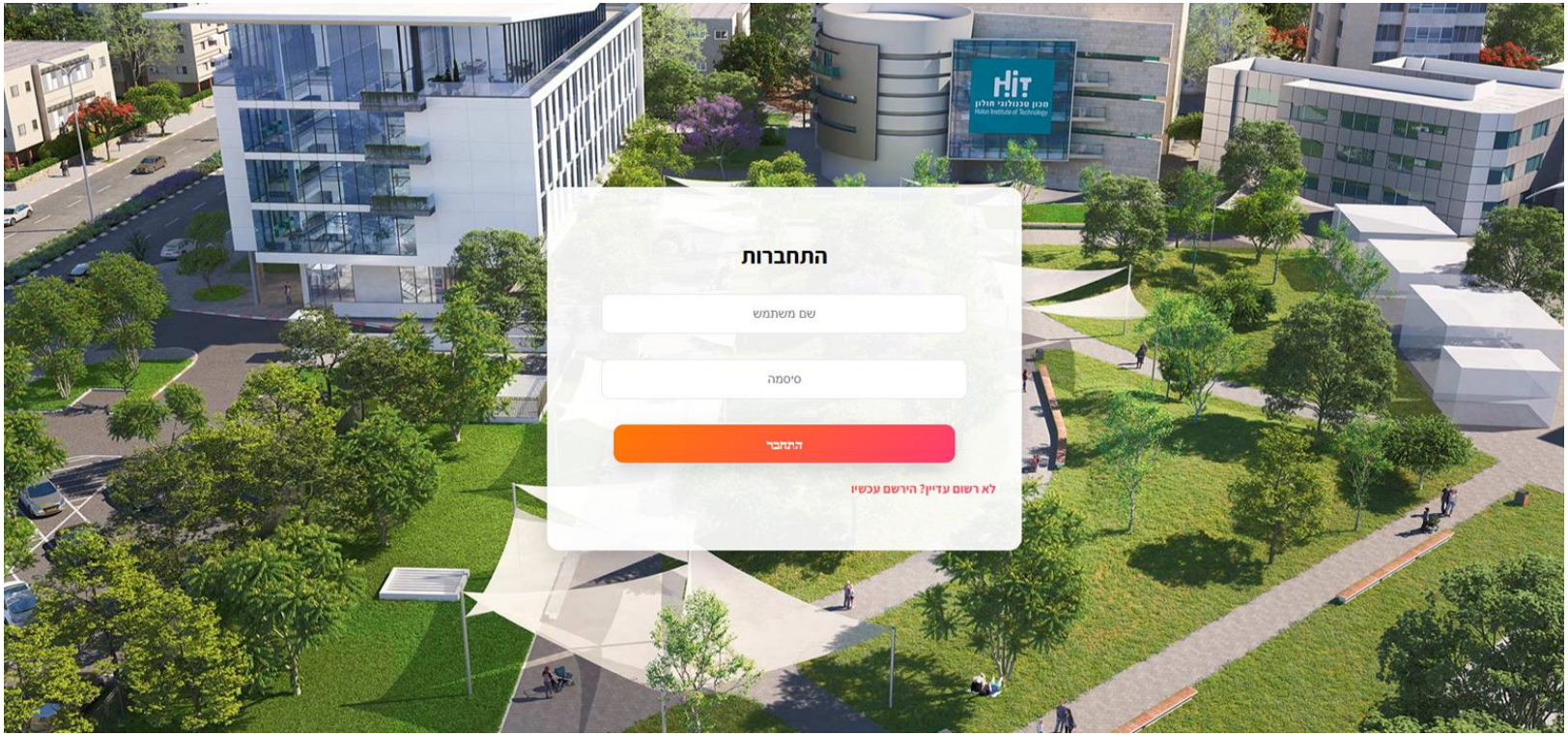
- Add Area page

- Add shade page

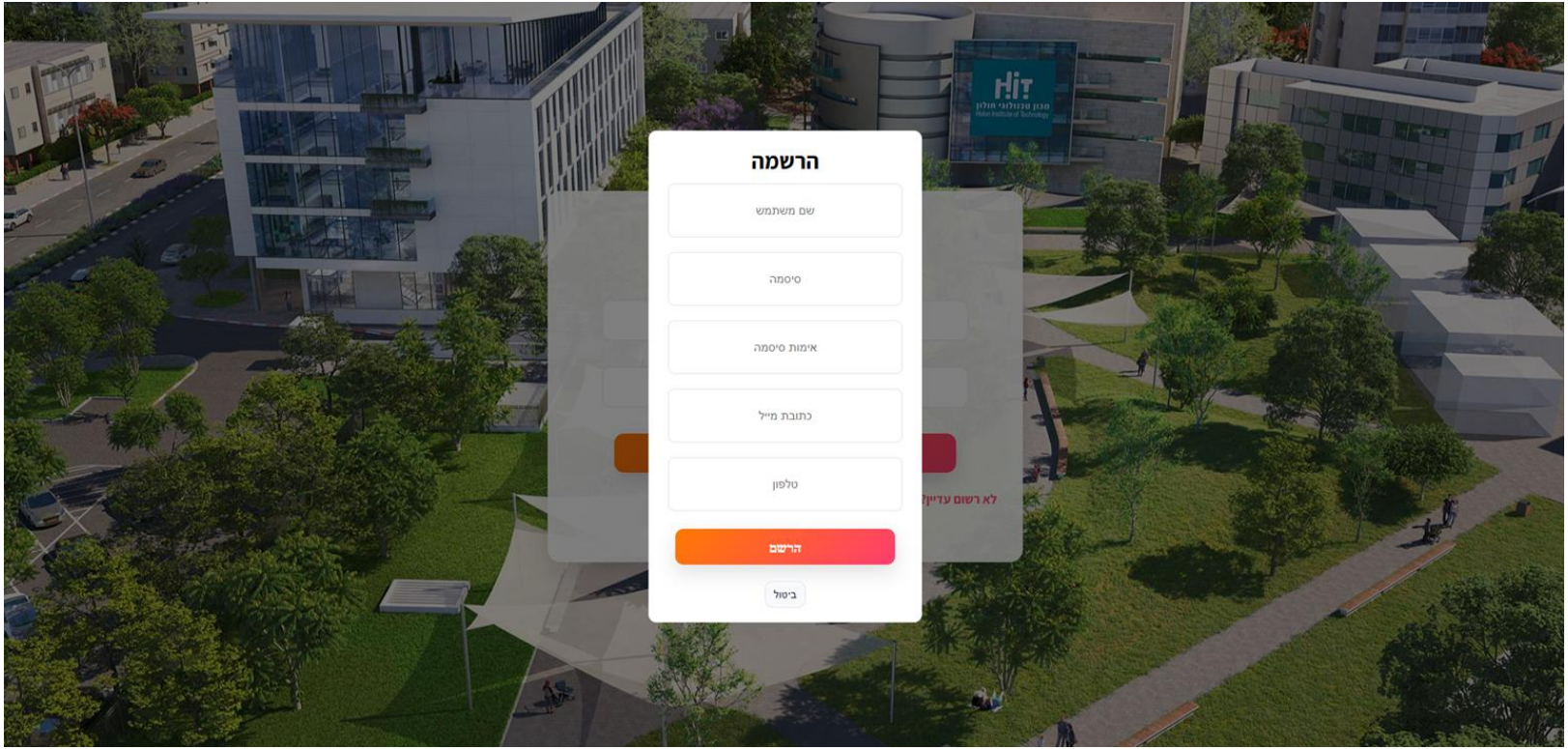
- Area’s shades control

- User Management & Roles

Login page

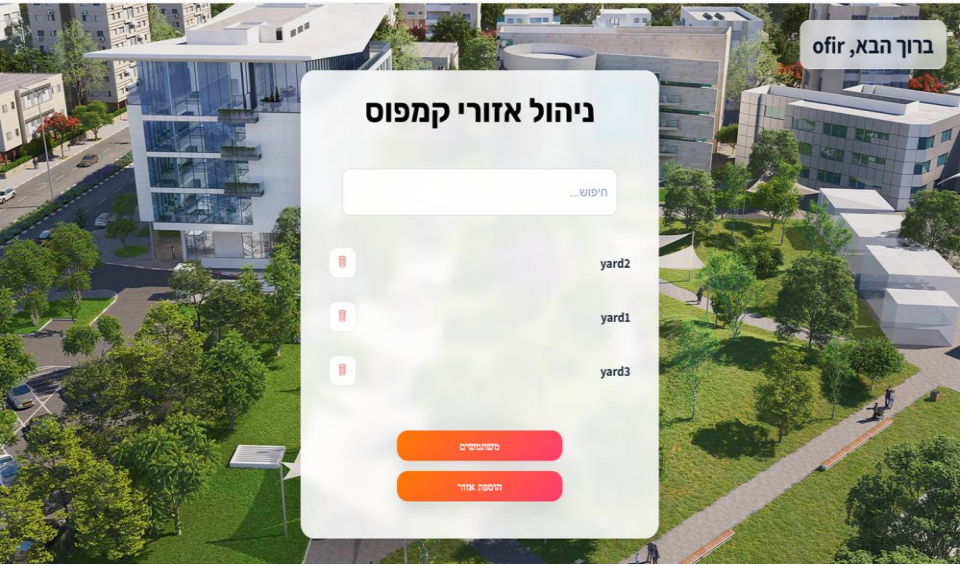

HIT
מכון טכנולוגי חולון
Holon Institute of Technology
התחברות
שם משתמש
סיסמה
התחבר
לא רשום עדיין? הירשם עכשיו

Sign up

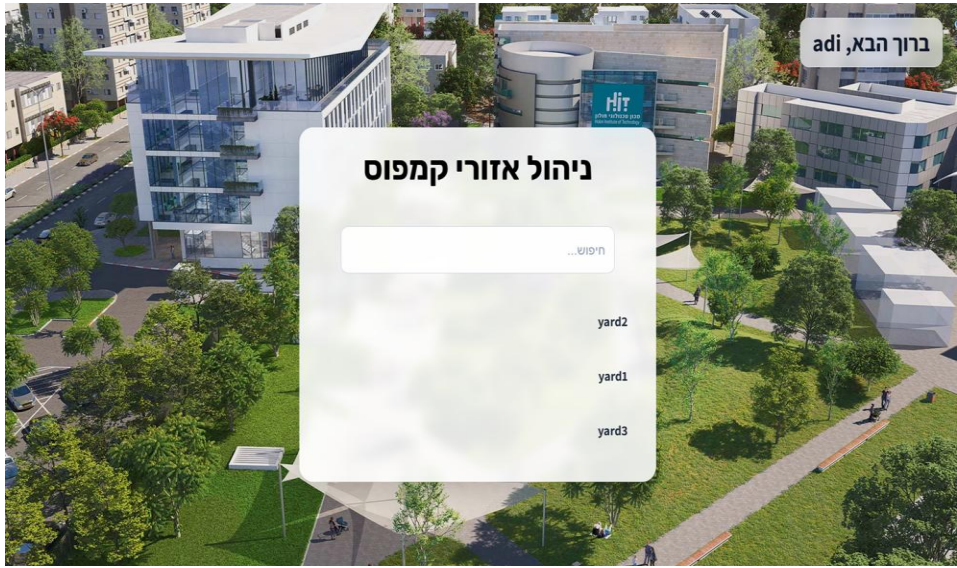

הרשמה
שם משתמש
סיסמה
אימות סיסמה
כתובת מייל
טלפון
הרשם
ביטול

Areas management page

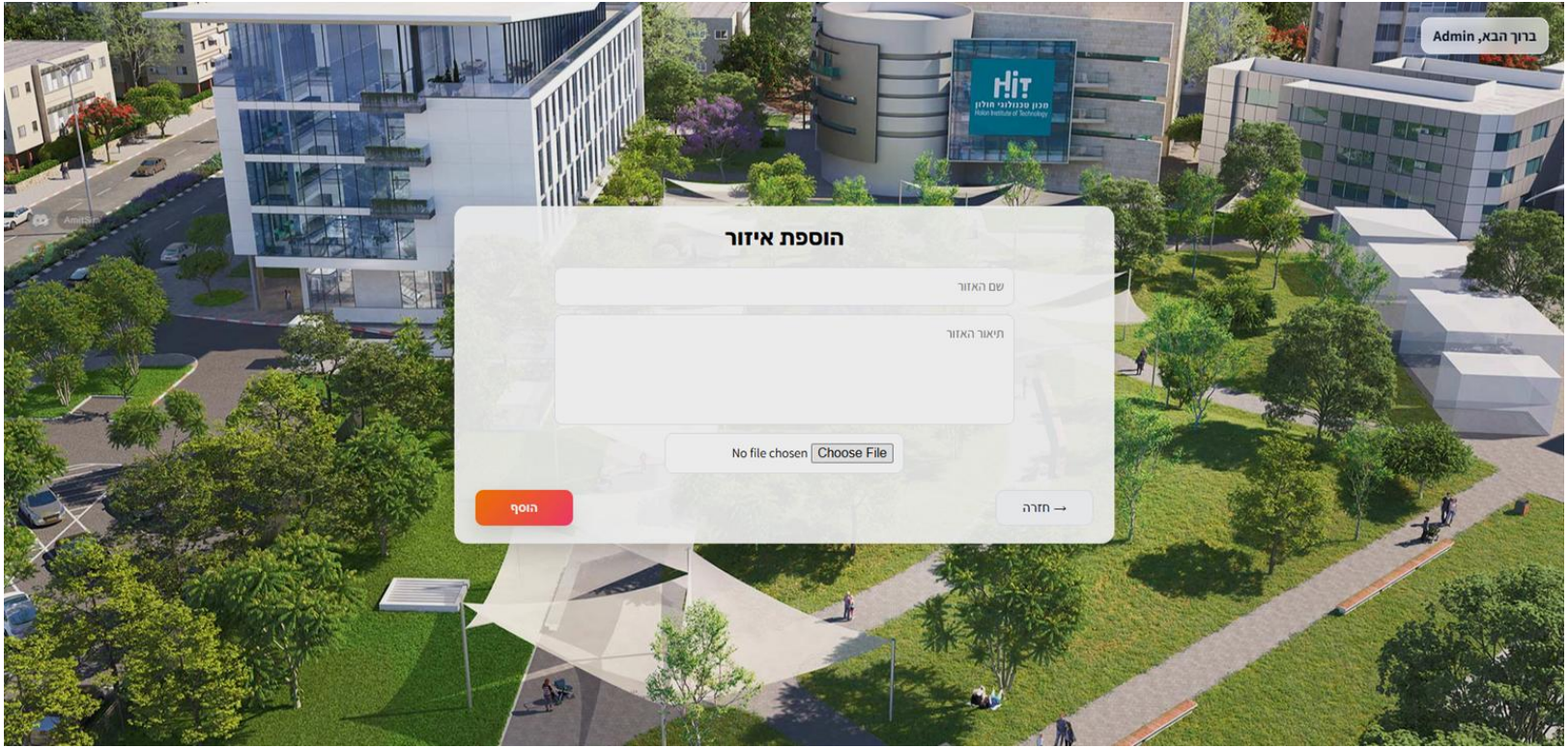
Admin



Viewer



Add area page



Add shade page

Admin

yard 1

מחוץ לבניין 8

עריכת אזור | הוספת הצללה

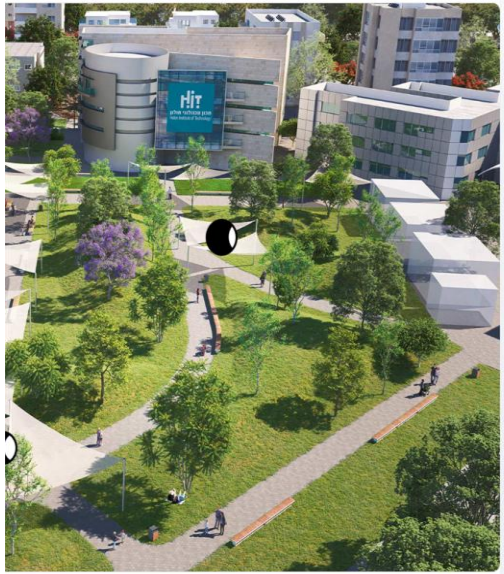
הצללות

מספר ההצללות באזור - 2

שם הצללה	אחוז	מחיקה
צלייה 1	27%	
צלייה 2	55%	

חזרה

Viewer



HITyard 1

תמונה כללית של המכללה

הצללות

מספר ההצללות באזור - 2

שם הצללה	אחוז	מחיקה
צלייה 1	50%	–
צלייה 2	70%	–

חזרה

Area's shades control - Edit area

שם האזור

yard 1

תיאור האזור

מחוץ לבניין 8

שמור בטל

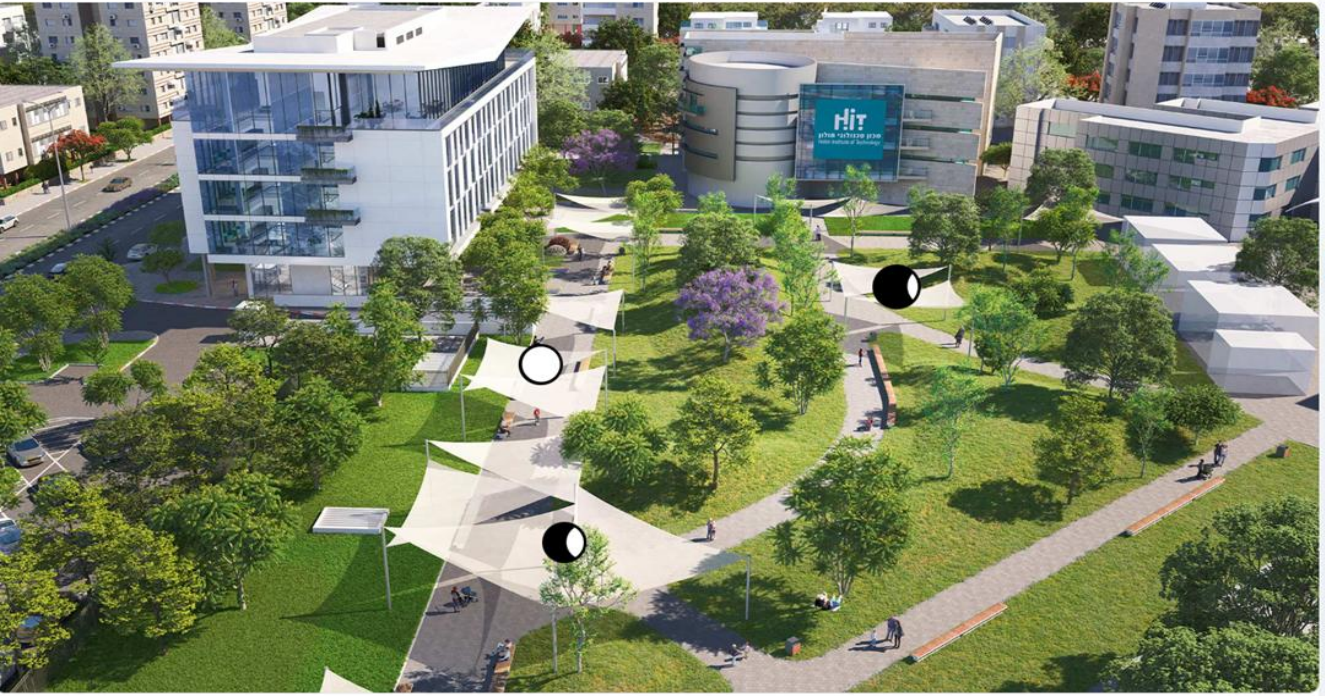
הצללות

מספר ההצללות באזור – 2

שם הצללה	אחוז	מחיקה
צלייה 1	27	
צלייה 2	55	

חזרה

Area's shades control - Add shade



HITyard 1

תמונה כללית של המכללה

אחוז	תיאור
761	534
64	64

שמור | בטל

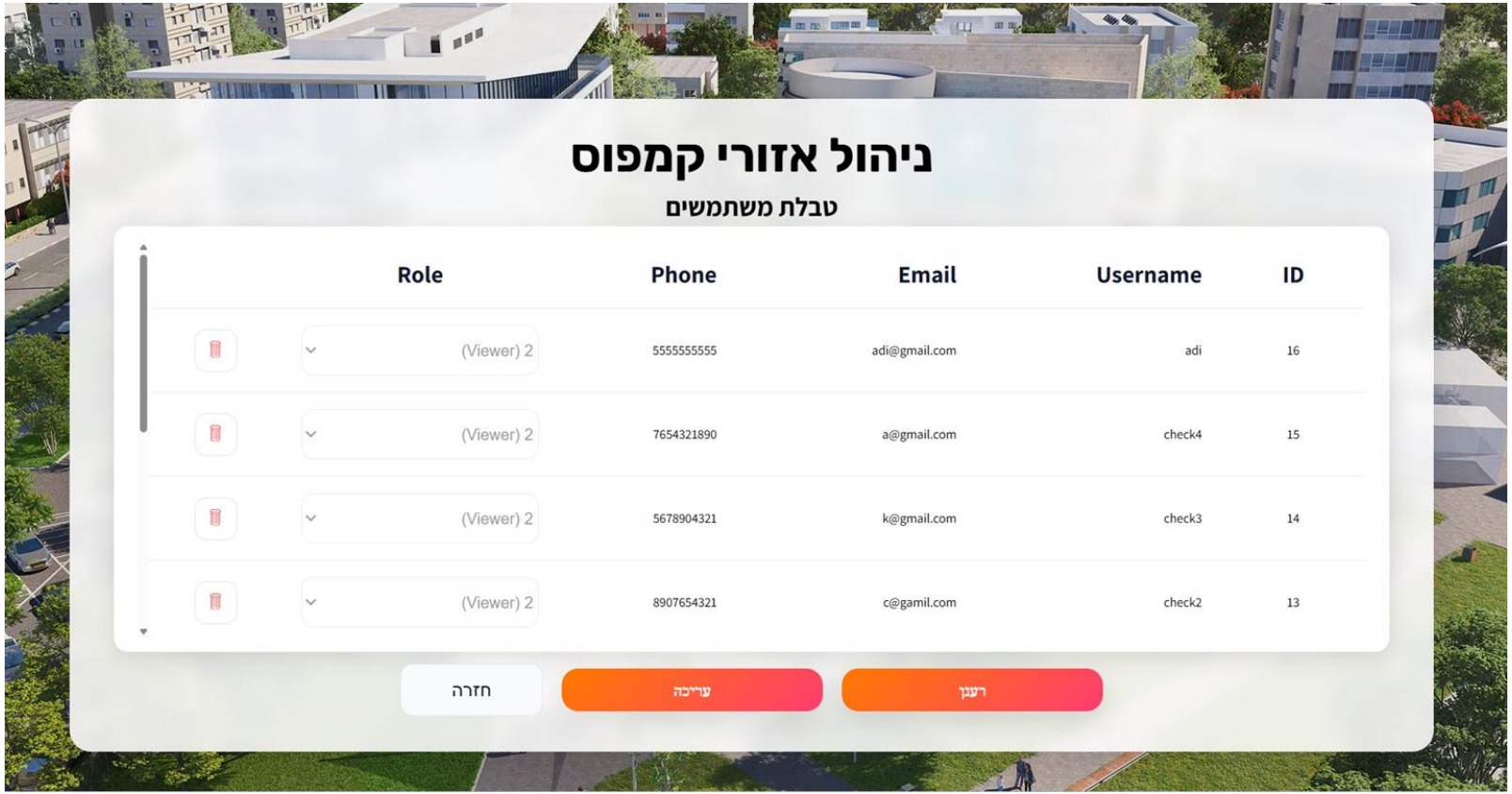
הצללות

מספר ההצללות באזור – 2

שם הצללה	אחוז	מחיקה
צלייה 1	50%	
צלייה 2	70%	

חזרה

Admin-Only: User Management & Roles



Unexpected problems

- Creating the database of the project and updating it throughout the development.
- Understanding how to work simultaneously on the project without holding each-other back.
- Making adaptations for the app to fit mobile and pc users alike required additional efforts and considerations.
- Adjusting the app to new requirements that surfaced during developments while still maintaining its current qualities.

Coping and solving

- **Database creation and adaptations:**
  Invested time into learning all the necessary concepts and practices for creating, maintaining and updating database. We broke each task down to small pieces so we can carefully and successfully create the optimal database.
- **Working simultaneously**:
  We studied Git in order to understand how to work correctly with Branches, Pull and Push requests and held meetings in order to sync with each other and to prepare for the next sprint.

Coping and solving

- **Adapting for new requirements and changes:**
  To handle changing requirements, we ran a quick gap analysis (“what works / what must change”), prioritized the items, and implemented them in small PRs bit by bit. We used Git feature branches with a protected main and regular syncs to stay aligned.
- **Adapting the app to mobile users:**
  We redesigned and optimised the UI to be responsive and friendly to both PC and Mobile users. We tested the app on all the popular screen sizes and worked on comprehensive solutions for the problems we faced.

Technologies Used

➢ **Frontend – React**

- Delivers a responsive and dynamic user interface
- Provides real-time interaction with the system through REST APIs
- Ensures accessibility from any device or browser

➢ **Backend – Node.js (Express)**

- Handles server-side logic and API requests efficiently
- Connects seamlessly with the database
- Enables secure data management and user authentication

➢ **Database – MySQL (MySQL Workbench)**

- Stores structured data such as shade locations, dimensions, and maintenance logs
- Provides reliable data integrity and fast query performance
- Supports relationships between users, areas, and historical records

Conclusions for further developments

- Git is a mandatory part of the development process.
- Setting clear schedules and following them is crucial.
- Distribution of work between each team member.
- Tests/Feedback -> Adaptations loop - testing and collecting feedback for new and existing features and adjusting the app accordingly was a great way to reach the best version of the app we could.